Iteration 4 — Verification Report

The wheel corrections hold up, the robot was over the speed limit until now, and the lab door failed for a reason worth knowing

Summary

The corrections are confirmed, by two independent measurements

Straight line: over a taped 7.620 m, odometry reported 7.6261 m and 7.6123 m — a scale error of **0.08% and 0.10%**. `wheel_radius: 0.175` is right.

Turning: the gyro/wheel yaw scale went from **1.062** to **0.987 and 1.004**. The 6.2% is gone.

The radius hypothesis in the revision report was correct, and the physical re-measurement agrees with the predicted 0.1752 m to within 0.2 mm.

Three things came out of the new bags that were not the point of the test, and all three matter more than the thing that was.

what	why it matters
The robot has been over the IGVC speed limit for four iterations.	With `wheel_radius` 6.1% low, a commanded 2.235 m/s was really **2.371 m/s = 5.30 mph**. The correction fixes this as a side effect. Section 3.
The recovery round robin does not reset between goals.	Directly observed: run 1's recovery started at `Wait`, run 2's started at `BackUp`. The rung you get depends on how many recoveries have run since `bt_navigator` started. Section 6.
The IMU lag got worse even though the rate tripled.	19.6 → 55.5 Hz is live, but the measured lag went 24.8 → 40–54 ms. Not burstiness — measured, and the stream is clean. Unexplained. Section 4.

No files were changed this round. The two parameters I am permitted to touch are now correct, and the one defect worth fixing needs a design decision and a multi-goal test I cannot run from bags. Section 8 says what I would change and why I did not do it unilaterally.

1. Straight-line test: the radius is confirmed

25 ft = **7.6200 m**, base_link (the drive axle) from mark to mark. Net displacement is the measurement, because the tape measured two points; path length is shown beside it because the runs were not perfectly straight and a wandering run inflates path length while net displacement still matches.

run	source	net	path	truth / net	heading change
real_bag1	wheel odometry	7.6261 m	7.6351 m	**0.9992**	–0.63°
real_bag1	EKF (local)	7.6343 m	7.6415 m	0.9981	–0.25°
real_bag2	wheel odometry	7.6123 m	7.7397 m	**1.0010**	+3.97°
real_bag2	EKF (local)	7.6179 m	7.7430 m	1.0003	+3.68°

The two runs bracket unity from opposite sides, which is what measurement noise looks like rather than a residual scale error. For comparison, the old `wheel_radius` of 0.165 would have reported **7.185 m** for the same 7.620 m — 5.7% short. Note bag 2's path length exceeds its net by 12.7 cm, consistent with the +3.97° of wander; that is a property of the driving, not of the odometry.

2. Turning test: the yaw scale is fixed

Net heading change is a bad estimator here — a 0.05 deg/s gyro bias adds 2.75° over a 55 s run, which swamps the few degrees of net rotation, and the transport lag biases a naive regression toward zero. Two estimators were used instead, both after removing the gyro bias measured on stationary stretches and after aligning the two signals by cross-correlation:

run	bias removed	lag	slope (gyro/wheel)	r^2	$\int\lvert\omega\rvert$ ratio	total rotation
real_bag2	+0.0000 rad/s	50 ms	0.9527	0.956	0.9690	47° *(straight-line test, thin)*
real_bag3	–0.0012 rad/s	40 ms	**0.9876**	0.991	0.9914	164°
real_bag4	–0.0012 rad/s	40 ms	**1.0014**	0.994	0.9971	253°

An independent estimator in `it4_freshness.py`, which uses a different resampling and a different peak refinement, gives 0.9865 and 1.0040 on the same two bags — agreement to 0.3%, so this is not an artefact of one method.

Where that leaves the geometry

Best-conditioned run (bag 4, 253° of rotation, r^2 0.994): **1.0014**, a 0.14% error. Bag 3: 0.9876, a 1.24% error the other way. Call the residual **0.5% ± 0.9%**.

That is at or below the noise floor of this measurement. There is nothing left here worth chasing, and in particular **no reason to touch** `wheel_separation_multiplier` — leave it at 1.0.

The 2.5 mm disagreement between the URDF (0.8524 m) and the control config (0.85487 m) is **0.29% of the track**. Only the control config's value feeds odometry, so it does not affect any of this; the URDF's copy only reaches TF and the footprint. Your instinct that it does not matter is right, with one caveat in section 7.

3. The robot has been over the speed limit since iteration 1

This is the most consequential thing in this report

With `wheel_radius` configured at 0.165 and truly 0.175, a commanded `max_vel_x` of 2.235 m/s (exactly 5 mph) set a wheel speed of 2.235/0.165 = 13.55 rad/s, which at the true radius is **0.175 × 13.55 = 2.371 m/s = 5.30 mph**.

Every run in iterations 1 through 4 was capable of **6.1% over the IGVC limit**, and the robot's own logs could never have shown it, because the odometry was scaled by the same wrong constant.

The correction fixes this automatically: 2.235/0.175 = 12.77 rad/s, true speed 2.235 m/s. **No change to** `max_vel_x` **is needed.** It is worth knowing it happened, in case any earlier run was ever submitted or timed.

4. The IMU rate change is live, and the lag got worse anyway

`update_rate: 60` is built. The IMU now publishes at **51.5–55.5 Hz**, up from 19.55–19.62. Mean sample age therefore falls from 25.5 ms to about 9 ms, and the EKF gets 2.8× as many corrections. That part worked.

The lag did not follow:

quantity	iteration 4 (19.6 Hz)	now (55.5 Hz)	reading
IMU gyro behind wheel odometry	24.8 ms	**40.4–48.7 ms**	Worse by roughly 16 ms.
EKF output behind wheel odometry	46.6 ms	43.8–53.1 ms	Essentially unchanged — and this is the one the controller actually reads.
Header-stamp age at bag receipt	—	0.1 ± 0.1 ms (IMU)	The driver stamps on receipt, so this measures nothing about transport. Noted so it is not mistaken for evidence.

The obvious hypothesis was serial congestion at 115200 baud — a link near capacity delivers samples in clumps, which raises the mean age of the newest sample without changing the average rate. **Measured, and it is not that.** Inter-arrival spacing is **18.01 ms median, 20.13 ms at p95, with zero gaps below half the nominal period** in every bag. The stream is clean and evenly spaced; 1–2.4% of gaps exceed 150% of nominal, which is occasional dropped samples, not bunching.

So this is unexplained, and I am not going to guess

Two things worth checking before the next run, in this order:

• **Did the `latency_timer` fix survive?** It is a udev rule on the FTDI device. A re-enumeration or a reinstall can lose it. Read back `/sys/bus/usb-serial/devices/ttyUSB1/latency_timer` and confirm it is still 1. This is the single most likely cause of a ~16 ms regression.

• **Is the UM7 applying more internal filtering at the higher output rate?** That would show as group delay rather than transport delay, and would not be visible in arrival spacing.

Also note the comparison is across different motion profiles, so the two numbers are not strictly like for like. The direction is consistent across three bags, which is why it is reported rather than dismissed.

The `BAUD_115200` hard-code in `src/um7` remains the structural fix if headroom ever is the problem — but on this evidence it is not currently the problem, so it has dropped down the priority list rather than up.

5. Why the lab door failed, with the numbers

Run 2 (bag 4) aborted with error 208, NO_VALID_PATH, after three planning failures. Taking the local cost map frame at the last failure (t = 33.66 s), with the robot at (–6.53, –3.62) heading 237.6° and the goal 1.25 m away at (–7.33, –4.58):

The corridor is 1.10 m at its narrowest

distance along the direct line	free left	free right	gap
0.00 m	0.625	0.675	1.300 m
0.40 m	0.625	0.600	1.225 m
0.60 m	0.625	0.475	**1.100 m**
0.80 m	0.575	0.675	1.250 m
1.00 m	opens out	0.650	2.650 m

Measured to **lethal** cells, not inflated ones, so this is the obstacle itself. The footprint is 0.980 m wide. The door therefore has **12 cm of total slack, 6 cm per side**, and only if the robot is perfectly square to it.

And the robot cannot be perfectly square to it

Support width of this footprint across the corridor axis, by heading error:

heading error	0°	5°	10°	15°	20°	25°	30°
width needed	0.980	1.013	1.038	1.072	1.140	1.200	1.250 m

At a 1.100 m gap the robot must stay within about **17°** of the corridor axis. The course to the goal is 230.1°. The lattice (`diff_2.5cm.json`) has 16 non-uniform headings, and the two nearest to that course are **225°** and **243.4°** — 5.1° and 13.3° off. Holding 225° costs 1.013 m of the 1.100 m available, leaving **4.3 cm per side**; holding 243.4° needs about 1.09 m and leaves essentially nothing.

The answer to your question

The door was wide enough for the robot to **fit**. It was not wide enough for the robot to **fit while being quantised**.

The planner must thread 0.980 m of body through a 1.100 m gap using poses on a 2.5 cm grid at one of 16 fixed headings, the best of which is 5.1° off the corridor axis and eats 3.3 cm of the 12 cm of slack. What remains is 4.3 cm per side on a 2.5 cm grid — one or two viable lateral positions, which every primitive in the chain must land on.

This is a legitimate NO_VALID_PATH. The search really did exhaust. It is not a tuning failure and not a bug.

Two supporting details

observation	what it says
The robot could hold only **6 of 16** lattice headings at its own position (45, 63, 207, 225, 243, 270°), and **5 of 16** at the goal (63, 90, 117, 135, 225°)	225° is free at both ends, so a solution is not geometrically excluded — which is why this is a margin-and-quantisation failure rather than an impossible goal.
A 0.50 m turning radius displaces the robot sideways by 0.030 m for 20° of turn and 0.067 m for 30°	Any meaningful course correction inside the throat spends the entire remaining margin. The robot has to be lined up *before* it enters.
The goal region holds 1448 footprint-free poses of 19984 tested (7.25%), and the exact goal is free at 68°	The goal itself was fine. Nothing to fix at the goal end.

One caveat that may change the arithmetic

The 1.100 m is the gap *as the cost map saw it*. The revision report measured obstacle paint at **5–14 cm thick** where a flat wall should be one or two 2.5 cm cells, from the unreferenced heading drift in the odom frame.

If the smear is costing 5–10 cm here, the real doorway is nearer **1.15–1.20 m** and the robot had roughly twice the margin the planner believed. **Worth taping the actual door** — if it measures well over 1.10 m, that is a direct measurement of the smear and strengthens the case for fusing an absolute heading reference.

6. The recovery round robin does not reset between goals

This one is a genuine defect, and it is visible by direct comparison of the two autonomous runs. The round robin in `recovery_nav_bt.xml` is ordered `Wait, BackUp(0.30), Wait, BackUp(0.50), Wait, ClearLocal, ClearEverything`.

run	first recovery rung that ran	expected
real_bag3 (run 1)	`Wait`, 2.02 s, robot moved 0.022 m	child[0] — correct
real_bag4 (run 2)	`BackUp`, 3.72 s, robot reversed 0.311 m at 0.107 m/s	child[0] again — **it started at child[1]**
real_bag4, second episode	`Wait`, 2.02 s	child[2] — consistent with having started at [1]

`RoundRobinNode` advances `current_child_idx_` on any non-RUNNING child result and resets it only in `halt()`. BehaviorTree.CPP only halts a node that is currently RUNNING, and after a recovery returns SUCCESS the round robin is already IDLE — so when the tree is halted at goal completion, this node is skipped and its index survives into the next goal.

Why this matters more than it looks

The whole point of the iteration-4 reordering was that **the first rung is a no-op**: stand still for 2 s, let STVL's in-frustum decay drop a transient mark, and never risk a reverse that cannot start. That guarantee only holds on the first goal after launch.

Worse, the last two rungs clear the cost maps. A leaked index can therefore make **clearing the cost map the first recovery action of a new goal** — precisely the thing the brief says must only ever be a last resort.

The index leaks by one per recovery episode, so a long session drifts steadily toward the destructive end of the list.

A related oddity I could not fully pin down

Run 2 aborted after only **two** recovery episodes, although `number_of_retries` on `NavigateRecovery` is **8**. The behavior tree log shows no gate or round-robin events at the third failure at all, which means `RecoveryNode` returned FAILURE from its `retry_count_ < number_of_retries_` check rather than ticking the recovery branch. `RecoveryNode::halt()` does reset `retry_count_`, and it is called explicitly when the first child succeeds — so this should not accumulate the way the round-robin index does. I can construct a story where it does, but not confirm it from these bags. **Reported as an observation, not a diagnosis.**

The one unambiguous good news in this section

The iteration-4 BackUp fix works. Run 2's reverse completed: 0.311 m at a peak of 0.107 m/s over 3.72 s, returning SUCCESS. Across iterations 1 to 4 every single BackUp aborted with error 714 before moving. This is the first recovery reverse this project has ever completed.

Dropping `backup_speed` to 0.10, which shrank the `isCollisionFree` veto window from 0.25 m to 0.10 m, is confirmed as the right call.

7. Smaller things in the new bags

observation	detail
Stale `cmd_vel` stamps are real and now visible	`diffbot_base_controller` logs *"Ignoring the received message ... older than the current time by 0.51–0.99 s"* in bursts of ten during recovery, in both autonomous runs. The velocity smoother republishes on its own clock while carrying its input's stamp, so a held command ages past `cmd_vel_timeout` 0.5. Harmless here — the robot should be stopped anyway — but it confirms the loose end flagged in the original report, and it means the last command keeps being resent rather than being withdrawn.
`/plan` is below its configured rate	`RateController` is set to 2.0 Hz; measured 1.57 Hz (bag 3) and 0.92 Hz (bag 4). Bag 4's figure is explained by the planner failing and the tree leaving the branch; bag 3's is not, and is worth a look if replanning latency ever matters again.
The footprint is now **8.6 mm per side** conservative	Outer tire face from the corrected numbers is 0.85487/2 + 0.10795/2 = **0.48141 m**; the configured footprint still uses **0.49**. That is 1.7 cm of total width, which at the lab door would have been **40% more margin** at the throat. See section 8 for why I did not simply change it.
The URDF's derived half width is now self-inconsistent	`base_width/2 + wheel_yoff + wheel_len/2` = 0.3048 + 0.1214 + 0.05398 = **0.48018 m**, but the comment above it still reads *"Half width 0.486 m at the outer face of the tires"*. Stale comment, no functional effect.
`/speed_governor/info` typo is fixed	Recording cleanly at 10 Hz. `/behavior_tree_log` is now recorded too, and it is what made section 6 possible — please keep it.
`costmap_raw` is still not recorded	Fourth iteration carrying this. It is the map the behavior server actually reads for its collision checks, and without it clearance has to be measured on a 6.4 Hz proxy.

8. What I changed, and what I deliberately did not

> **No files were changed this round**
>
> `Course2027.yaml` and `recovery_nav_bt.xml` are untouched. The parameters in them are now measured-correct, and the two things worth doing both have reasons not to do them blind.

candidate change	why not, this round
Shrink the footprint half width from 0.49 to **0.48141**	Defensible — 0.49 came from the old, wrong geometry rounded up, not from a deliberate margin — and it would have bought 40% more margin at the lab door. But it sets the planner's idea of the body to the **exact physical outline with zero margin**, on a robot that has been coming within 2.5 cm of lethal cells, and the standing goal is to stay as far from obstacles as possible. Shrinking a safety dimension on my own judgement is not my call. **Say the word and it is a one-line change.**
Restructure the round robin so a leaked index is harmless	The obvious fix — prefix every rung with its own Wait — does not actually solve it, because a leaked index can still land directly on a cost map clear. A real fix has to stop relying on round-robin ordering for the last-resort property, which is a design change, and verifying it needs a multi-goal run that bags cannot give me. Section 9 has the options.
Touch `wheel_separation_multiplier`	No. The residual is 0.5% ± 0.9%, at the noise floor. Leave it at 1.0.
Lower `max_vel_x` for speed compliance	Not needed. The radius correction already fixed it; 2.235 m/s is now genuinely 5.00 mph.

9. Recommendations, in order

#	action	where	why
1	Confirm the `latency_timer` is still 1 on the IMU port	`/sys/bus/usb-serial/devices/ttyUSB1/latency_timer`	Most likely cause of the 16 ms lag regression, and it costs one `cat`.
2	Tape the actual lab doorway	a measurement	If it is well over 1.10 m, that difference *is* the cost map smear, measured directly — which is worth more than the door.
3	Decide the footprint question	`Course2027.yaml`	0.49 against a true 0.48141. Your call on margin; I will make the change either way.
4	Fix the round-robin leak	`recovery_nav_bt.xml`, and possibly a node	Options: accept it and make every rung safe to run first (drops the clears from the round robin entirely); or restart `bt_navigator` between runs as a stopgap; or carry a tiny patch to `RoundRobinNode`. Needs your preference before I write it.
5	Consider fusing absolute yaw from `scan_matcher`	`Local_EKF.yaml`	Unchanged from the revision report, and section 5's caveat raises its value: the smear may be what closed the door.
6	Reconcile the URDF's 0.8524 against the control config's 0.85487	`diffbot_description.urdf.xacro`	2.5 mm, no effect on odometry, but the footprint and TF derive from the URDF copy. Fix the stale 0.486 comment while you are there.
7	Record `costmap_raw`	the bag command	Fourth request. Everything else in the command is now right.

10. Tools added

tool	what it does
`it4y_straight.py`	Straight-line odometry scale against a taped distance, reporting net displacement and path length separately so a wandering run is not mistaken for a scale error.
`it4y_yawscale.py`	Yaw-rate scale with gyro bias removed from stationary stretches and the transport lag removed by cross-correlation, fitted through the origin on actively turning samples, with an integral-of-$\|\omega\|$ cross-check.
`it4y_imuburst.py`	Inter-arrival spacing of the IMU stream, to test whether a serial link is delivering in clumps.
`it4y_whynopath.py`	Decomposes a NO_VALID_PATH into its three possible causes: an unreachable goal region, a too-narrow corridor, or no room to turn. Reports free lattice headings at both ends.
`make_verif_report_it4y.py`	Builds this document.

Four bags analysed: real_bag1 and real_bag2 (straight line, 25 ft), real_bag3 (autonomous run 1, succeeded), real_bag4 (autonomous run 2, aborted at the lab door). No source files were changed.